Machine Learning

Tabular data in the wild — Lecture 20 (Chapter 9, Part 3)

Amir Farbin

Part 3 — Tabular data in the wild

Where we are

- Lecture 19: scikit-learn on SUSY, one clean file we know well. Lecture 21: a first network in Keras
- Today: **real tables** we don't know, the way your Kaggle project will go: get the data, check it, look at it, prepare it, establish a **baseline**, compare methods
- The data: a **benchmark** of 59 tables (a fixed set of datasets everyone tests methods on), as a Kaggle competition, `bench-tab-v1`
- Lecture 22 puts the Keras template on one of these tables

Which method for a table?

- Grinsztajn, Oyallon & Varoquaux (2022): on **medium-sized tables of high-level features** (quantities someone already built), **tree-based models** beat deep learning
- **Decision tree:** a sequence of yes/no cuts on single features. **Random forest:** many trees on random parts of the data, averaged. **Boosting:** small trees added one after another, each fixing the last one's errors
- It depends on the **regime**. With lots of data, computing and **low-level features** (raw measurements), a deep network learns its own features: the SUSY paper. Little data: boosted trees
- For your tabular project: **boosted decision trees** are the method to beat (`HistGradientBoostingClassifier`, XGBoost)

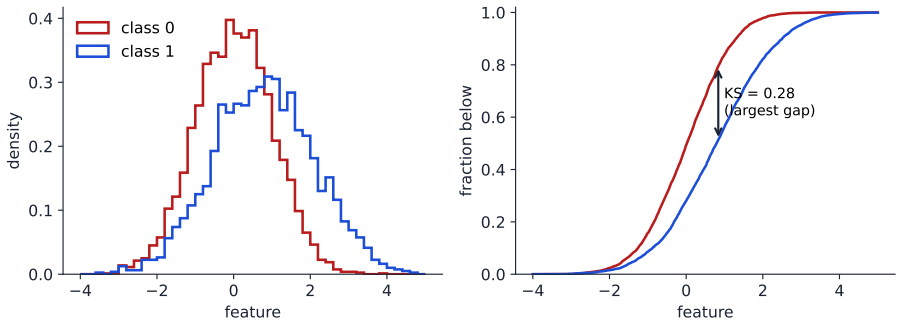
Getting the data, reproducibly

- Download by **command**, kept in the notebook: `wget`, `curl`, `kaggle competitions download`
- An **API** (application programming interface): the commands, and their arguments, a service accepts. Kaggle's is a **REST** API: a request is a URL, the reply is data, usually **JSON**. The `kaggle` **CLI** (command-line interface) wraps it
- **Read the folder before the documentation:** `data_train` + `target_train` are inputs and answers (a **supervised** problem); `data_test` has none: that's what Kaggle scores
- **Parquet:** binary, column by column, with names and types. pandas reads it; you can't `cat` it

Due diligence: don't trust anybody

- Check every dataset yourself, and early: rows match, missing values, feature counts, which are categorical. Make a guess, test it, move on
- Including your own code: a rule of thumb called 18 datasets "multiclass". The benchmark's own task file says they are **regression**
- Know what the columns **mean**. The credit dataset: limit, sex, age, six months of repayment status, bills and payments
- `glob` returns files in the filesystem's order: pick datasets **by name**

Look at every feature, per class



- **Kolmogorov–Smirnov (KS):** the largest gap between the two **cumulative distributions** (the fraction of values below each point): 0 = identical, 1 = no overlap, independent of binning
- A quick score for each feature alone. On credit default: **repayment status** 0.22–0.37, payments and credit limit 0.12–0.18, bill amounts under 0.05

Correlations

- The **correlation coefficient** ρ: move one feature by one standard deviation, and the other moves by ρ of its own
- $|\rho| \approx 1$: the same information twice. Keep one (**feature selection**); traditional methods do better with fewer features
- Too many features for a colour map: **flatten, sort by** $|\rho|$, read the top of the list
- Classes can share every histogram and still **differ in their correlations**: Fisher uses that

Categorical features

category	label encoded	one-hot encoded		
		red	green	blue
red	0	**1**	0	0
green	1	0	**1**	0
blue	2	0	0	**1**
green	1	0	**1**	0

implies an order:
blue > green > red

no order:
one weight per category

- **Ordinal or categorical?** If shuffling the labels loses nothing, it's **categorical**; if the order means something, **ordinal**. Months late, age: ordered. It's a judgement call
- **One-hot encoding:** one 0/1 column per category. A linear model makes one cut per feature, so binning a measurement (young, middle-aged, old) can help it
- `fit` learns the mapping, `transform` applies it. **Keep the encoder** (and pickle it)

Scaling, and checking

- Why: features from -2 to hundreds of thousands, and optimizers work badly when ranges differ
- `StandardScaler`: subtract the mean, divide by the standard deviation. `MinMaxScaler`: to $[0, 1]$
- Fit **once**, on training data, and reuse the same instance everywhere. Fitting on everything **leaks** test information into training
- **Check after scaling.** A `drop` whose result was never assigned kept 8 columns that should have gone: `df = df.drop(columns=...)`

A baseline, then a comparison

- **Baseline first:** a simple, fast model. LDA on credit default, AUC **0.78**. If a fancy model does worse, something is wrong
- Then many classifiers in one loop: polymorphism, all share `fit` and `score` (**accuracy**: the fraction right; 0.5 is chance for balanced classes). A **pipeline** chains scaler and classifier
- In class the loop scored the bare classifier on **unscaled** data: everything near chance. Fixed: electricity 0.74–0.82; eye_movements 0.50–0.57, a hard dataset
- The baseline said 0.78. A table of 0.50s should have been a red flag

Your project

- **By Wednesday, November 4:** a Kaggle tabular classification competition, picked from the guide's list or agreed with me
- Not every CSV is a table of examples. **Simple:** one row per example. **Many tables** linked by IDs: join and summarize first. **Not really tabular:** game-play mouse capture, time series, event logs: summarize per example first
- The steps are today's: load, check, histogram per class, clean and scale, baseline, compare. A **non-trivial** result (clearly better than chance and the baseline), not a winning one
- Hyper-parameters, the settings you choose (`C`, `gamma`, `n_neighbors`), are what the comparison loop searches

Vocabulary

- **benchmark, curated, Kaggle competition, API, REST, JSON, CLI, Parquet, supervised**
- **decision tree, random forest, boosting / boosted decision trees, regime, high-level vs low-level features**
- **due diligence, glob order, Kolmogorov–Smirnov statistic, cumulative distribution**
- **correlation coefficient, feature selection**
- **categorical vs ordinal, one-hot encoding, `fit` / `transform`, encoder**
- **standardize, min-max scaling, data leakage**
- **baseline, accuracy, chance, pipeline, polymorphism, hyper-parameter, non-trivial result**

What comes next

- Lecture 22: the Keras template from Lecture 21 on one of these tables, through a **module** of today's functions
- Lab 9: your own networks on SUSY. And your Kaggle dataset, by November 4